

FUNÇÕES E ARQUIVOS

Prof. André Backes

FUNÇÕES

C vs C++ | Funções

- Funções funcionam da mesma forma em C e C++
 - Tanto as funções convencionais quanto as recursivas
 - No entanto, C++ possui alguns recursos extras

Linguagem C

```
int Square(int a){
    return (a*a);
}

int main (){
    int n1, n2;
    printf("Entre com um numero: ");
    scanf("%d",&n1);
    n2 = Square(n1);
    printf("O seu quadrado vale: %d",n2);

    return 0;
}
```

Linguagem C++

```
int Square(int a){
    return (a*a);
}

int main (){
    int n1, n2;
    cout << "Entre com um numero: ";
    cin >> n1;
    n2 = Square(n1);
    cout << "O seu quadrado vale: " << n2;

    return 0;
}
```

C vs C++ | Funções

- C++ não permite que uma função seja declarada dentro de outras funções

```
void escreve(int a) {  
    int Square(int a) {  
        return (a*a);  
    }  
  
    cout << Square(a) << endl;  
}
```

C vs C++ | Funções

- Podemos definir valores padrão para parâmetros da função
 - Parâmetros com valor padrão devem vir sempre por último
 - O parâmetro se torna opcional, ou seja, se não for definido o valor padrão será usado

```
float reajuste(float salario, float juros = 0.10){  
    return salario + salario * juros;  
}  
  
int main(){  
  
    cout << reajuste(1000,0.25) << endl;  
    cout << reajuste(1000) << endl;  
  
    return 0;  
}
```

C vs C++ | Funções

- Tipos de passagem de parâmetros
 - Passagem de parâmetros por valor
 - Uma cópia do dado é feita e passada para a função
 - Passagem de parâmetros por referência utilizando ponteiros
 - Passamos para a função os endereços das variáveis na memória
 - Passagem de parâmetros por referência utilizando o operador de referência &

Passagem de parâmetros usando referência &

- Lembrando: uma referência é uma variável que funciona como um sinônimo para outra variável já existente

```
void incrementa(int &n){  
    n = n + 1;  
    cout << "Dentro da funcao: x = " << n << endl;  
}  
  
int main (){  
    int x = 5;  
    cout << "Antes da funcao: x = " << x << endl;  
    incrementa(x);  
    cout << "Depois da funcao: x = " << x << endl;  
  
    return 0;  
}
```

Passagem de parâmetros usando referência &

- Arrays são passados usando ponteiros ou referências
 - Se usar referência, a função apenas aceitará arrays que possuam aquele tamanho

```
void imprime(int *ar, int n){  
    for(int i=0; i<n; i++)  
        cout << ar[i] << endl;  
}
```

```
int main(){  
    int vet[5] = {1,2,3,4,5};  
    imprime(vet,5);  
  
    return 0;  
}
```

```
void imprime(int (&ar)[5]){  
    int n = sizeof(ar) / sizeof(ar[0]);  
    for(int i=0; i<n; i++)  
        cout << ar[i] << endl;  
}
```

```
int main(){  
    int vet1[5] = {1,2,3,4,5};  
    int vet2[3] = {1,2,3};  
  
    imprime(vet1);  
    imprime(vet2); /// ERRO!  
  
    return 0;  
}
```


Sobrecarga de função

- A linguagem C++ permite criar várias funções com o mesmo nome
 - Precisam ter parâmetros diferentes
 - Mecanismo de prototipagem frequentemente utilizado na programação orientada a objetos

```
void imprime(double A){  
    cout << "double: " << A << endl;  
}  
  
void imprime(int A){  
    cout << "int: " << A << endl;  
}  
  
void imprime(string A){  
    cout << "string: " << A << endl;  
}  
  
int main(){  
    imprime(10);  
    imprime(5.1);  
    imprime("Ana");  
  
    return 0;  
}
```

Sobrecarga de função

- O tipo de retorno da função não é considerado para determinar a função a ser chamada
 - Não é possível criar duas função com o mesmo nome e a mesma lista de parâmetros, mesmo que o tipo de retorno delas seja diferente

```
int imprime(int A){  
    cout << "double: " << A << endl;  
    return 0;  
}
```



```
void imprime(int A){  
    cout << "int: " << A << endl;  
}
```

Sobrecarga de função

- Esse recurso deve ser utilizado com cuidado
- Caso não exista uma função implementada com os tipos dos parâmetros desejados
 - O compilador irá procurar a função cujos parâmetros sejam os mais adequados e, para tanto, fará uso de conversão de tipos e *casting*
 - Isso pode gerar problemas como funções sendo chamadas de forma incorreta ou dar margem para ambiguidades

Sobrecarga de função

- Neste caso, não existe a implementação da função para um parâmetro do tipo **double**

```
void imprime(int A){  
    cout << "int: " << A << endl;  
}  
  
void imprime(string A){  
    cout << "string: " << A << endl;  
}  
  
int main(){  
    imprime(10);  
    imprime(5.1);  
    imprime("Ana");  
  
    return 0;  
}
```

Funções em linha

- A linguagem C++ permite solicitar ao compilador faça uma expansão “em linha” da função
 - Ao invés de gerar o código para chamar a função, o compilador irá inserir o corpo completo da função em cada lugar do código em que a função for chamada
- Forma geral

```
inline tipo nome_função(parâmetros) {  
    sequência de declarações e comandos  
}
```

Funções em linha

- Trata-se de um recurso de otimização
 - Visa melhorar o tempo e o uso da memória durante a execução do programa
 - Tem a desvantagem de talvez aumentar o tamanho final do programa
 - É um recurso normalmente utilizado para funções que executam com muita frequência

"Função convencional"

```
int maior(int x, int y){  
    return (x>y)?x:y;  
}
```

```
int c = maior(a,b);
```

"Função em linha"

```
inline int maior(int x, int y){  
    return (x>y)?x:y;  
}
```

```
int c = maior(a,b);
```

```
int c = (a>b)?a:b;
```

Funções em linha

- Funções “em linha” se parecem muito com funções macro
 - Porém com a vantagem de possuir segurança baseada em tipo (*type-safety*)
 - Chamadas de macro não fazem verificação de tipo nos parâmetros nem verificam se eles são bem formados, enquanto uma função exige isso

Funções em linha

- Restrições e limitações
 - Não existe garantia de que o compilador conseguirá expandir uma função *inline* recursiva
 - Não é adequada em funções com número de parâmetros variável
 - Deve-se evitar o uso de **goto**
 - Deve-se evitar o uso de funções aninhadas

ARQUIVOS EM C++

Manipulando arquivos

- A linguagem C++ possui uma série de classes e métodos para a manipulação de arquivos na biblioteca **fstream**
 - Suas funções se limitam a operações de abrir/fechar e ler/escrever caracteres e bytes
 - É tarefa do programador criar a função que irá ler ou escrever um arquivo de uma maneira específica

Manipulando arquivos

- A linguagem C++ possui 3 classes disponíveis para manipular arquivos:
 - **ofstream**: representa um arquivo de saída
 - **ifstream**: representa um arquivo de entrada
 - **fstream**: representa um arquivo aberto tanto para a leitura quanto para a escrita de dados

Abrindo um arquivo

- Para abrir um arquivo, usa-se o método **open()**

```
objeto.open(nome_arquivo, modo) ;
```

- O método **open()** recebe
 - **nome_arquivo**: string contendo o nome do arquivo que deverá ser aberto
 - **modo**: uma ou mais flags contendo o modo de abertura do arquivo (opcional)

Abrindo um arquivo

- No parâmetro **nome_arquivo** pode-se trabalhar com caminhos absolutos ou relativos
 - Caminho absoluto: descrição de um caminho desde o diretório raiz
 - C:\\Projetos\\dados.txt
 - Caminho relativo: descrição de um caminho desde o diretório atual
 - dados.txt
 - ../alunos.txt

Abrindo um arquivo

- O modo determina o uso será feito do arquivo

Modo	Função
ios::in	Abre o arquivo para leitura. O arquivo deve existir.
ios::out	Abre o arquivo para escrita. Cria o arquivo se não existir. Apaga o anterior se ele existir.
ios::binary	Abre o arquivo no modo binário. Se não for especificado, o arquivo é aberto no modo texto.
ios::app	Abre o arquivo e move a posição de escrita para o fim do arquivo ("append"). Cria o arquivo se não existir.
ios::ate	Abre o arquivo e move a posição de leitura/escrita para o fim do arquivo. Diferente do modo <code>ios::app</code> , esse modo permite editar outras regiões do arquivo.
ios::trunc	Se o arquivo for aberto para escrita e já existir, seu conteúdo será apagado.

Abrindo um arquivo

- Para saber se um arquivo foi aberto com sucesso utilizamos o método **is_open()**

`objeto.is_open()`

- Esse método retorna:
 - **true**: arquivo aberto com sucesso
 - **false**: erro na abertura do arquivo

Abrindo um arquivo

```
#include <fstream>
#include <iostream>

using namespace std;

int main() {
    //Abre o arquivo usando o construtor
    ofstream arq1("dados1.txt");

    //Abre o arquivo usando o construtor
    //e especificando o modo
    ofstream arq2("dados2.txt", ios::out | ios::binary);

    //Abre o arquivo usando o método open()
    ofstream arq;
    arq.open("dados.txt");

    if(arq.is_open()) {
        cout << "Arquivo aberto com sucesso!";
        arq.close();
    } else {
        cout << "Erro ao abrir o arquivo!";
    }

    return 0;
}
```


Fechando um arquivo

- Sempre que terminamos de usar um arquivo que foi aberto, devemos fechá-lo
- Para isso usamos o método **close()**

```
arquivo.close();
```

- Uma vez fechado, podemos usar o mesmo objeto para abrir um novo arquivo

Fechando um arquivo

- Por que precisamos fechar o arquivo?
 - Ao fechar um arquivo, todo dado que tenha permanecido no "buffer" é gravado
 - O "buffer" é uma região de memória que armazena temporariamente o conteúdo a ser gravado em disco
 - Apenas quando o "buffer" está cheio é que seu conteúdo é gravado no disco

Fechando um arquivo

- Mas por que utilizar um “buffer”? Eficiência!
 - Para ler e escrever arquivos no disco temos que posicionar a cabeça de gravação em um ponto específico do disco
 - Fazer isso para cada caractere lido/escrito, tornaria a leitura/escrita de um arquivo muita lenta
 - Assim a gravação só é realizada quando há um volume razoável de dados ou quando o arquivo for fechado

Fechando um arquivo

```
#include <fstream>
#include <iostream>

using namespace std;

int main() {
    //Abre o arquivo usando o construtor
    ofstream arq1("dados1.txt");

    //Abre o arquivo usando o construtor
    //e especificando o modo
    ofstream arq2("dados2.txt", ios::out | ios::binary);

    //Abre o arquivo usando o método open()
    ofstream arq;
    arq.open("dados.txt");

    if(arq.is_open()) {
        cout << "Arquivo aberto com sucesso!";
        arq.close();
    } else {
        cout << "Erro ao abrir o arquivo!";
    }

    return 0;
}
```

Escrevendo em arquivo texto

- Para escrever dados é comum usarmos a classe **ofstream**
 - Também podemos usar classe **fstream**
 - O processo é igual ao usado para a escrita de dados usando o objeto `cout`
- Forma geral:

```
arquivo << dados;
```

- Podemos também escrever diversas variáveis em sequência:

```
arquivo << dados_1 << dados_2 << ... << dados_N;
```

Escrevendo em arquivo texto

```
#include <fstream>
#include <iostream>
using namespace std;
int main(){
    string nome = "Andre";
    int idade = 38;
    float altura = 1.74;

    ofstream arq;
    arq.open("dados.txt");
    if(arq.is_open()){
        arq << nome << endl;
        arq << idade << endl;
        arq << altura << endl;

        arq.close();
    }else
        cout << "Erro ao abrir o arquivo!" << endl;

    return 0;
}
```

Escrevendo em arquivo texto

- Assim como o **cout**, o objeto arquivo possui vários métodos que permitem manipular os dados

Método	Função
arquivo.put(char &ch)	grava o caracter armazenado em ch
arquivo.write(char *str, int n)	grava os n primeiros caracteres da string str
arquivo.precision(int n)	especifica que n caracteres devem ser gravados após o ponto decimal
arquivo.flush()	força a gravação do buffer de dados
arquivo.width(int n)	define a largura da saída
arquivo.fill(char ch)	define o caractere ch para ser usado para preencher a largura de saída

Lendo de arquivo texto

- Para ler dados é comum usarmos a classe **ifstream**
 - Também podemos usar a classe **fstream**
 - O processo é igual ao usado para a leitura de dados usando o objeto cin

- Forma geral:

```
arquivo >> dados;
```

- Podemos também ler diversas variáveis em sequência:

```
arquivo >> dados_1 >> dados_2 >> ... >> dados_N;
```


Lendo dados de arquivo texto

```
#include <fstream>
#include <iostream>
using namespace std;
int main() {
    string nome;
    int idade;
    float altura;

    ifstream arq;
    arq.open("dados.txt");
    if(arq.is_open()){
        arq >> nome;
        arq >> idade;
        arq >> altura;

        arq.close();

        cout << "Nome: " << nome << endl;
        cout << "Idade: " << idade << endl;
        cout << "Altura: " << altura << endl;
    } else
        cout << "Erro ao abrir o arquivo!" << endl;

    return 0;
}
```

Lendo dados de arquivo texto

- Assim como o **cin**, o objeto arquivo possui vários métodos que permitem manipular os dados

Método	Função
arq.bad()	retorna se houve uma falha na leitura
arq.fail()	retorna se a leitura é compatível com o esperado
arq.clear()	limpa o estado das variáveis de erro do buffer
arq.eof()	retorna se o fim do arquivo foi encontrado
arq.get(char &ch)	lê um caractere do arquivo e armazena em ch
arq.read(char *buffer, int n)	Lê n bytes e armazena em buffer
arq.putback(char c)	recoloca o último caractere lido de volta no buffer

Escrevendo em arquivo binário

- Para escrever dados é comum usarmos a classe **ofstream** (ou a classe **fstream**) com o modo de abertura **ios::binary**
- A escrita é feita utilizando o método `write()`

```
arquivo.write(const char* buffer, streamsize N);
```

- onde
 - `buffer`: ponteiro para a região que contém os dados
 - `N`: número de bytes a ser gravado no arquivo

Escrevendo em arquivo binário

- Esse método trabalha com blocos de memória de qualquer tipo
 - int, float, double, array, struct ...

```
#include <fstream>
#include <iostream>
using namespace std;
int main() {
    ofstream arq;
    arq.open("dados.txt", ios::binary | ios::out);
    if(arq.is_open()) {
        //grava o valor de x no arquivo
        float x = 5;
        arq.write((char*)&x, sizeof(x));

        //grava todo o array no arquivo (5 int)
        int v[5] = {1,2,3,4,5};
        arq.write((char*)v, 5*sizeof(int));

        arq.close();
    } else {
        cout << "Erro ao abrir o arquivo!" << endl;
    }

    return 0;
}
```

Lendo de arquivo binário

- Para ler dados é comum usarmos a classe **ifstream** (ou a classe **fstream**) com o modo de abertura **ios::binary**
- A leitura é feita utilizando o método **read()**

```
arquivo.read(char* buffer, streamsize N);
```

- onde
 - buffer: ponteiro para a região que armazenará os dados
 - N: número de bytes a ser lido do arquivo

Lendo dados de arquivo binário

- Esse método trabalha com blocos de memória de qualquer tipo
 - int, float, double, array, struct ...

```
#include <fstream>
#include <iostream>
using namespace std;
int main(){
    ifstream arq;
    arq.open("dados.txt",ios::binary | ios::in);
    if(arq.is_open()){
        //lê o valor de x do arquivo
        float x;
        arq.read((char*)&x, sizeof(x));
        cout << "x = " << x << endl;
        //lê todo o array do arquivo (5 posições)
        int v[5];
        arq.read((char*)v, 5*sizeof(int));
        for(int i=0; i< 5; i++)
            cout << v[i] << endl;

        arq.close();
    }else
        cout << "Erro ao abrir o arquivo!" << endl;

    return 0;
}
```

Lendo um arquivo até o final

- Para verificar se chegamos ao final de um arquivo usamos o método **eof()** associado ao arquivo, cujo protótipo é:

```
bool eof();
```

- Esse método retorna **true** se o final do arquivo já foi atingido

Lendo um arquivo até o final

- Importante
 - O método **eof()** testa o indicador de fim de arquivo e não o próprio arquivo
 - Assim, deve-se evitar o seu uso para terminar um loop
- Mas qual o problema disso?
 - Isto significa que outra função (como as de leitura) é responsável por alterar o indicador para indicar que o final foi alcançado

Lendo um arquivo até o final

```
int main(){
    ifstream  arq;
    arq.open("matriz.txt");
    if(arq.is_open()){
        int v, soma = 0;
        while(1){
            arq >> v;
            if(arq.eof())
                break;
            soma = soma + v;
        }
        cout << "Soma = " << soma << endl;
        arq.close();
    }else
        cout << "Erro ao abrir o arquivo!" << endl;

    return 0;
}
```

Material Complementar

- Aula 11 - Função, passagem de parâmetros e valores padrão
 - <https://youtu.be/qO2mxOhb68E>
- Aula 12 - Sobrecarga de função
 - <https://youtu.be/8-1IU1bLNaE>
- Aula 13 - Funções em linha
 - <https://youtu.be/YSxCZa8Rvw0>
- Aula 17 - Arquivos: Abrir e Fechar
 - https://youtu.be/_VOvi0JeQ1w
- Aula 18 - Arquivos: Escrevendo e lendo em arquivo texto
 - <https://youtu.be/BqmGWsnuHP0>
- Aula 19 - Arquivos: Escrevendo e lendo em arquivo binário
 - <https://youtu.be/ptKKwAkkWTU>
- Aula 20 - Arquivos: Lendo um arquivo até o final
 - <https://youtu.be/dxet-NKgYNE>
- Aula 22 - Sobrecarga de função e operador delete
 - <https://youtu.be/HlcmHxA3pU>